

Design Explanation

1. Meta-data design

- a. Meta-data will be stored in the first block of the directory. The first 16 bytes will hold a random salt for PBKDF2, which will be used to derive secure encryption and authentication keys from the user's password. The next 4 bytes will hold the PBKDF2 iteration count, which is the number of times the hashing process will be repeated when deriving a key. The next byte will be the username length, so that we know when the username ends, since the username will be at most 128 bytes. The next 128 bytes will hold the username, padded with zero bytes. The next 16 bytes will hold the random IV. The next 16 bytes will hold the metadata tag, in the form of a truncated HMAC-SHA256 tag. The last 843 bytes will hold the encrypted secret metadata. In the plaintext metadata, the file length will be held in the first 4 bytes, the next 32 bytes will hold a value derived from the password that will be used for password checking, and the last 807 bytes will be random padding.

2. User authentication:

- a. The password isn't actually stored in the file system, but password-related information is stored in a derived form. During file creation, a random 16 byte salt and a PBKDF2 iteration count are stored in the metadata block. When the user enters their password, the system will apply the hashing process using the password, salt, and iteration count to derive a key. This key will then be used to derive an AES encryption key and an HMAC authentication key. During authentication, the system will derive the two keys from the entered password, verify the metadata integrity using the HMAC tag, and compare the password verification value, stored in the encrypted metadata, to the new value using the entered password. If the two values match, access is granted, but a

PasswordIncorrectException is thrown if the values are different and no data is accessed.

3. Encryption design

- a. each file will be stored as a directory, with each block holding 1024 bytes. Block 0 will hold all metadata, and blocks 1-N will store the encrypted file content. Each of those blocks will hold the 16 byte IV, the first 16 bytes of a HMAC-SHA256 tag, and 992 bytes of our payload. A fresh IV is generated every time a block is rewritten, which is important because CTR mode becomes insecure if the same IV is reused with the same key for different plaintexts. Since an adversary can observe multiple versions of a file, reusing an IV would allow them to XOR two versions and recover information about the plaintext differences. Since each block is encrypted independently with its own IV and authenticated with an HMAC tag, the design remains secure even if an adversary can read every stored version of the file over time. The adversary will only learn the number of blocks used.

4. File length hiding

- a. The exact file length will be stored in the encrypted metadata section of block 0 of the file. Since the file data is stored in physical blocks of 1024 bytes, the number of blocks will reveal the file size range. This is unavoidable without increasing the number of physical blocks, which increases storage overhead. To avoid leaking additional information besides the number of blocks, any unused bytes in the final data block are filled with random padding before encryption. Since these bytes are encrypted, it will not be possible to distinguish them from valid ciphertext, preventing an attacker from determining where the plaintext ends in the final block.

5. Message authentication

- a. Implemented using HMAC-SHA256. Each encrypted file and its metadata block has a HMAC computer over the ciphertext (for file blocks) or the encrypted secret metadata (for the metadata block). The key for the HMAC is derived from the user password using PBKDF2 with a random salt. The HMAC ensures

integrity because the system verifies the HMAC before decrypting the content, preventing an attacker from tampering with encrypted blocks. The HMAC also ensures authenticity because when writing or cutting a file, the system recomputes the HMAC for the changed blocks, including the metadata, so that any future read can detect tampering.

6. Efficiency

- a. Each file is divided into fixed-size blocks. Only the blocks required to store the actual file data plus the single metadata block are allocated. Unused bytes in the last block are padded with random bytes that are still encrypted, allowing us to hide the exact file length, minimizing the number of physical files. Reads and writes access only the blocks affected, so modifying a single byte updates at most two blocks, the data block and potentially the metadata block. Max storage efficiency could be achieved by using very large blocks to reduce the number of physical files, but this would increase the number of bytes accessed and encrypted for small reads/writes, reducing speed. Max speed could be achieved with very small blocks, but would increase storage overhead due to block count and file system metadata.

Pseudocode

Key Derivation

```
derive_keys(password, salt, iteration_count)
```

```
    Get master key from PBKDF2 function using password, salt, and iteration_count
```

```
    Retrieve encryption key and authentication key from the master key
```

```
    Return encryption key and authentication key
```

Password Verification

```
verify_password(file_name, password):
```

```
    Extract salt, iteration count, username length, username, IV, stored tag, and  
    encrypted metadata
```

Derive encryption key and authentication key from password, salt, and iteration count

Compute HMAC tag from HMAC_SHA256 function using authentication key
if computed KMAC tag != stored tag

FAIL (metadata was modified)

Decrypt secret metadata using encryption key and IV

Retrieve password check value

Compute new password check value using encryption key
if stored value != computed value

FAIL (wrong password)

Return file length, encryption key, and authentication key

create Function

CREATE(file_name, username, password)

create directory using file name

Generate random 16 byte salt

Set iteration count

Derive encryption key and authentication key using the password, generated salt, and iteration count

Set password check value using encryption key

Set file length to 0

Construct secret metadata with file length, password check value, and random padding (807 bytes)

Generate random IV

Encrypt secret metadata using the encryption key and IV

Construct entire metadata: salt, iteration count, username length, padded username, IV, and encrypted secret metadata

Generate HMAC tag using authentication key and the metadata

Add HMAC tag to metadata block

Save metadata to /file_name/0

length Function

LENGTH(file_name, password)

Retrieve metadata block

Call verify password function passing the metadata block and password

Retrieve file length if password matches

Return file length

read Function

READ(filename, start, len, password)

retrieve meta data block

Verify password using password and metadata block

Retrieve encryption key, authentication key, and file length

If $\text{start} + \text{len} > \text{file_length}$

FAIL

Calculate start and end block

result = empty buffer

For each block i:

retrieve ciphertext from block

verify MAC

decrypt ciphertext with IV and encryption key

Appended bytes to result

return result

check_integrity Function

CHECK_INTEGRITY(filename, password)

Retrieve metadata block

Verify password using the metadata block

Retrieve encryption key, authentication key, and file length

For each data block:

Read block

```
    Recalculate MAC
    If computed_MAC != saved_MAC:
        return false
    return true
```

cut Function

```
CUT(filename, new_length, password)
    Retrieve metadata block
    Verify password using metadata block
    Retrieve encryption key, authentication key, and file length
    If new_length > file_length:
        FAIL
    Determine last needed block
    Truncate final block contents
    Re-encrypt block
    Recompute MAC
    delete blocks after final block
    Update file length in secret metadata
    Re-encrypt secret metadata
    Recompute metadata tag
    Overwrite metadata block
```

Design Variations

1. Append-Only Writes
 - a. If writes only append to the file, we only have to encrypt the new blocks and update a running MAC, since existing blocks will be untouched. This improves speed and does not compromise security
2. Single-Version Disk Theft

- a. If adversaries can only read one version of the file, we can store the entire file as a single ciphertext with one IV and HMAC, minimizing storage and computation, while maintaining confidentiality and integrity.
3. CBC Mode
- a. Each block depends on the previous block's ciphertext, so random-access writes will be expensive, since modifying one block means having to decrypt all blocks before that block. Security will be stronger, but append-only writes will be a lot more efficient.
4. ECB Mode
- a. Since ECB encrypts each block independently, random-access reads and writes are fast, but it is not as secure because identical plaintext blocks will produce identical ciphertexts.

AuthInfo Class

1. This helper class was added to simplify how authentication and encryption keys are handled after password verification. Instead of having to recompute these values every time we call a function like read, write, etc., the AuthInfo object stores these values and returns them together. This allows the code to be much cleaner by grouping all authentication-related information in one object.